

# Python Básico

## III. Operadores

### Operadores lógicos

# Conclusões – Operadores Relacionais

- Operadores relacionais
- Operadores relacionais != atribuição
- Precedência de Operadores
- Operadores em objetos compostos nativos
- Operadores em objetos compostos módulos





# Operadores Lógicos



| Operador | Descrição                     | Resultado          | Código                       |
|----------|-------------------------------|--------------------|------------------------------|
| and      | Ambos verdadeiros, verdadeiro | True e True = True | (5 == 5) and (2 == 2) = True |

# Operadores Lógicos



| Operador | Descrição                | Resultado                 | Código                            |
|----------|--------------------------|---------------------------|-----------------------------------|
| and not  | Ambos verdadeiros, falso | True e não True = False   | (5 == 5) and not (2 == 2) = False |
|          |                          | True e não False = True   | (5 == 5) and not (2 != 2) = True  |
|          |                          | False e não True = False  | (5 != 5) and not (2 == 2) = False |
|          |                          | False e não False = False | (5 != 5) and not (2 != 2) = False |
| or not   | Um verdadeiro, falso     | True ou não True = True   | (5 == 5) or not (2 == 2) = True   |
|          |                          | True ou não False = True  | (5 == 5) or not (2 != 2) = True   |
|          |                          | False ou não True = False | (5 != 5) or not (2 == 2) = False  |
|          |                          | False ou não False = True | (5 != 5) or not (2 != 2) = True   |

# Operadores Lógicos



| Operador | Descrição | Resultado           | Código                                |
|----------|-----------|---------------------|---------------------------------------|
| in       | Presente. | True se existir     | 2 in [1,2,3,4] = True                 |
| in not   | Ausente.  | True se não existir | 5 not in [1,2,3,4] = True             |
| is       | É         | True se for igual   | obj_list_I is obj_list_II = False     |
| is not   | Não é     | False se for igual  | obj_list_I is not obj_list_II = False |

# Objetos Simples

and

```
1 print(f"{{(a == a) and (b == b)}}")
2 print(f"{{(a == a) and (b != b)}}")
3 print(f"{{(a != a) and (b == b)}}")
4 print(f"{{(a != a) and (b != b)}}")
```

[7] ✓ 0.0s

```
... True
   False
   False
   False
```

```
1 a = 4
2 b = 2
3
4 print(f"a == a: {{(a == a)}}")
5 print(f"b == b: {{(b == b)}}")
6 print(f"a != a: {{(a != a)}}")
7 print(f"b != b: {{(b != b)}}")
```

[9] ✓ 0.0s

```
... a == a: True
   b == b: True
   a != a: False
   b != b: False
```

```
1 print(f"{{(a == a) and not (b == b)}}")
2 print(f"{{(a == a) and not (b != b)}}")
3 print(f"{{(a != a) and not (b == b)}}")
4 print(f"{{(a != a) and not (b != b)}}")
```

[ ]

```
... False
   True
   False
   False
```

```
1 print(f"{{not (a == a) and (b == b)}}")
2 print(f"{{not (a == a) and (b != b)}}")
3 print(f"{{not (a != a) and (b == b)}}")
4 print(f"{{not (a != a) and (b != b)}}")
```

[8] ✓ 0.0s

```
... False
   False
   True
   False
```

Python



# Objetos Simples

or

```
1 print(f"{{(a == a) or (b == b)}}")
2 print(f"{{(a == a) or (b != b)}}")
3 print(f"{{(a != a) or (b == b)}}")
4 print(f"{{(a != a) or (b != b)}}")
```

[5] ✓ 0.0s

... True  
True  
True  
False

```
1 a = 4
2 b = 2
3
4 print(f"a == a: {{(a == a)}}")
5 print(f"b == b: {{(b == b)}}")
6 print(f"a != a: {{(a != a)}}")
7 print(f"b != b: {{(b != b)}}")
```

[9] ✓ 0.0s

... a == a: True  
b == b: True  
a != a: False  
b != b: False



```
1 print(f"{{(a == a) or not (b == b)}}")
2 print(f"{{(a == a) or not (b != b)}}")
3 print(f"{{(a != a) or not (b == b)}}")
4 print(f"{{(a != a) or not (b != b)}}")
```

[7] ✓ 0.0s

... True  
True  
False  
True

```
1 print(f"{{not (a == a) or (b == b)}}")
2 print(f"{{not (a == a) or (b != b)}}")
3 print(f"{{not (a != a) or (b == b)}}")
4 print(f"{{not (a != a) or (b != b)}}")
```

[6] ✓ 0.0s

... True  
False  
True  
True



# Objetos Compostos

and

```
1 print(f"{{(a == a) and (b == b)}}")
2 print(f"{{(a == a) and (b != b)}}")
3 print(f"{{(a != a) and (b == b)}}")
4 print(f"{{(a != a) and (b != b)}}")
```

[17] ✓ 0.0s

... True  
False  
False  
False

or

```
1 print(f"{{(a == a) or (b == b)}}")
2 print(f"{{(a == a) or (b != b)}}")
3 print(f"{{(a != a) or (b == b)}}")
4 print(f"{{(a != a) or (b != b)}}")
```

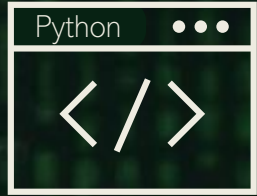
[20] ✓ 0.0s

... True  
True  
True  
False

```
1 a = ('a', 'b', 'c', 'd', 'e')
2 b = (1, 2, 3, 4, 5)
3 print(f"a == b: {{(a == b)}}")
4 print(f"a == a: {{(a == a)}}")
5 print(f"b == b: {{(b == b)}}")
```

[27] ✓ 0.0s

... a == b: False  
a == a: True  
b == b: True





# Numpy

Untitled61.ipynb ☆

Arquivo Editar Ver Inserir Ambiente de execução Ferramentas

Comandos + Código + Texto ▶ Executar tudo

[1] ✓ 0s

```
1 import numpy as np
2 a = np.array(('a', 'b', 'c', 'd', 'e'))
3 b = np.array((1, 2, 3, 4, 5))
4 print(f"a == b: {(a == b)}")
```

a == b: [False False False False False]

[3] ✓ 2s

1 !python --version && pip show numpy

... Python 3.12.13  
Name: numpy  
Version: 2.0.2



```
1 a = np.array(('a', 'b', 'c', 'd', 'e'))
2 b = np.array((1, 2, 3, 4, 5))
3 print(f"a == b: {(a == b)}")
```

[30] ✓ 0.0s

```
... a == b: False
/tmp/ipykernel_818/2970711146.py:3: FutureWarning: elementwise comparison failed; returning scalar instead, but in the future will perform elementwise comparison
  print(f"a == b: {(a == b)}")
```

```
1 !python --version && pip show numpy
```

[29] ✓ 0.7s

```
... Python 3.10.20
Name: numpy
Version: 1.23.5
```

# Numpy

Python



```
1 a = np.array((6, 7, 8, 9, 10))
2 b = np.array((1, 2, 3, 4, 5))
3 print(f"a == b: {(a == b)}")
4 print(f"a == a: {(a == a)}")
5 print(f"b == b: {(b == b)}")
```

[33] ✓ 0.0s

```
... a == b: [False False False False False]
a == a: [ True  True  True  True  True]
b == b: [ True  True  True  True  True]
```

```
1 print(f"{{(a == a) and (b == b)}}")
```

[35] ✖ 0.0s

```
-----
ValueError                                Traceback (most recent call last)
Cell In[35], line 1
----> 1 print(f"{{(a == a) and (b == b)}}")

ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()
```

```
1 print(f"{{(a == a) and (b == b)}}")
```

[3] ! 0s

```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_262/1518785975.py in <cell line: 0>()
----> 1 print(f"{{(a == a) and (b == b)}}")

ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()
```

# Numpy

numpy.all = and

numpy.any = or

```
1 a = np.array((6, 7, 8, 9, 10))
2 b = np.array((1, 2, 3, 4, 5))
3 print(f"a == b: {(a == b)}")
4 print(f"a == a: {(a == a)}")
5 print(f"b == b: {(b == b)}")
```

[33] ✓ 0.0s

```
... a == b: [False False False False False]
a == a: [ True  True  True  True  True]
b == b: [ True  True  True  True  True]
```



```
1 print(np.all((a == a, b == b)))
2 print(np.all((a != a, b == b)))
3 print(np.all((a == a, b != b)))
4 print(np.all((a != a, b != b)))
```

[25] ✓ 0.0s

```
... True
False
False
False
```

```
1 print(np.any((a == a, b == b)))
2 print(np.any((a != a, b == b)))
3 print(np.any((a == a, b != b)))
4 print(np.any((a != a, b != b)))
```

[26] ✓ 0.0s

```
... True
True
True
False
```

# Numpy

numpy.all = and

numpy.any = or



```
1 a = np.array([1, 2, 3])
2 b = np.array([1, 2, 3_000])
3 print(f"a == b: {(a == b)}")
```

[42]

✓ 0.0s

...

a == b: [ True True False]



```
1 c = np.array([4, 5, 6])
2
3 display(np.all((a == b, c == c)))
4 display(np.any((a == b, c == c)))
```

[46]

✓ 0.0s

...

False

...

True

Python





# Precedência de Operadores

| Operador                  |
|---------------------------|
| ( )                       |
| **                        |
| * / // %                  |
| + -                       |
| < <= > >=                 |
| == !=                     |
| = += -= *= /= //= %*= **= |
| and or                    |



# Conclusões

- Operadores lógicos
- Precedência de Operadores
- Operadores em objetos simples
- Operadores em objetos compostos nativos
- Operadores em objetos compostos módulos





Vamos  
praticar?

LIMC-IA



Vamos  
exercitar?

LIMC-1A





# Exercícios

01 – Dados as operações abaixo, qual o resultado da operação lógica:

$p \text{ and } q \text{ and } r \text{ and } s \text{ and } t$ ?

Como os operadores lógicos podem ser usados para inverter o resultado?

$p = 2.0 + 3.0 / 5.0 == 10$

$q = (p + 10.2) - 5.0 * 2.0 \leq 2.9$

$r = 10 \leq p / q + 33.8 < 100$

$s = 5 \neq p / (q + r)$

$t = 33 == s * 2$

Python



# Exercícios

02 – Usando apenas o operador or, qual seria o resultado final das operações abaixo? Como deve ser usado o not para inverter o resultado?

```
p = ('a',1,True,'b',0,False) != ('a',True,1,'b',False,0)
```

```
q = ['a',1,True,'b',0,False] != ('a',True,1,'b',False,0)
```

```
r = ['a',1,True,'b',0,False] != ['a',True,1,'b',False,0]
```

```
s = ['a',1,True,'b',0,False] > ['a',True,1]
```

```
t = ['a',1,True,'b',0,False] == 1
```

```
u = ['a',True,1] <= ['a',True,1] < ['a',1,True,'b',0,False]
```



## Exercícios

03 – No exercício 3 da aula 8 foi importado um arquivo .csv para um objeto da classe DataFrame pandas. Importe aqui o mesmo objeto e arredonde e edite o index como realizado no exercício 3 da aula 8. Usando operadores responda, a segunda linha é igual a terceira ou a terceira linha é igual a quarta?

04 – Usando operadores verifique se todos os valores contidos na primeira linha são maiores ou iguais a 10.

05 – Usando operadores verifique se todos os valores contidos na terceira coluna estão entre 0 inclusive e 40.





## Exercícios

06 – Usando operadores verifique se algum dos valores contidos na última linha está acima de 10.

07 – Recupere a DataFrame contendo apenas valores que estejam acima de 5 e abaixo de 7.

08 – Recupere a DataFrame contendo apenas valores que estejam abaixo de 5 inclusive ou acima de 7.

09 – Converta as duas últimas colunas do objeto DataFrame para dois objetos da classe numpy array. Algum dos valores contidos no primeiro array estão entre 2 inclusive e 5 inclusive?



## Exercícios

10 – Os valores contidos nos dois arrays são superiores à -1 e inferiores à 12?





# Python Básico



2 de abril de 2026